

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI LAUREA IN INGEGNERIA DELL'INFORMAZIONE

Realizzazione di un Dispositivo Domotico: Un Caso Studio con ESPHome

Relatore

Prof. Meneghini Matteo

Laureando

Sandrini Mattia

ANNO ACCADEMICO 2024-2025

Dedicato a Gaia, che mi ha insegnato che non esistono montagne troppo alte da essere scalate.

Dedicato al Nonno Bruno, che da lassù veglia sulla sua Famiglia con occhi paterni.

Sommario

Negli ultimi anni, la domotica ha assunto un ruolo sempre più centrale nella progettazione di ambienti residenziali e commerciali, offrendo soluzioni innovative per migliorare il comfort, la sicurezza e l'efficienza energetica delle abitazioni.

In questo contesto, sono nate piattaforme open-source come Home Assistant e EspHome. Home Assistant è una piattaforma di automazione che consente di gestire e integrare diversi dispositivi. EspHome, invece, è un framework progettato per configurare e programmare microcontrollori come ESP8266 e ESP32.

Il progetto presentato in questa tesi propone lo sviluppo di un dispositivo domotico multifunzione, basato su microcontrollore ESP32 programmato con EspHome e integrato con Home Assistant, pensato per essere installato nelle comuni scatole da incasso 503, sostituendo i classici interruttori e pulsanti a muro.

Questo dispositivo, dotato di vari sensori ambientali e pulsanti fisici, permette di monitorare le condizioni della stanza e di controllare direttamente i dispositivi intelligenti della casa, come luci, termostati, tapparelle. Grazie al display, l'utente può visualizzare in tempo reale dati rilevanti come temperatura, umidità, qualità dell'aria o altre informazioni.

Il risultato è una soluzione che combina l'estetica e la praticità dei classici frutti ad incasso con le potenzialità della domotica avanzata, rendendo l'interazione con la casa intelligente immediata e accessibile a tutti gli utenti, senza necessità di complessi interventi di modifica degli impianti preesistenti.

Indice

1	Introduzione	1
2	L'avvento della domotica	3
2.1	Definizione di domotica	3
2.2	Evoluzione storica	3
2.2.1	Il protocollo X10	3
2.2.2	Barriere economiche e rischio di insuccesso	4
2.2.3	La rivoluzione wireless: IEEE 802.11	4
2.3	Scenario di mercato e tendenze	4
2.3.1	Previsioni di mercato	4
2.3.2	Tendenze trainanti	4
2.3.3	Principali investitori	5
2.3.4	Sfide principali	5
2.3.5	Aspetti non-market	5
2.4	Sicurezza	5
2.4.1	Rischi specifici dei sistemi domotici	6
2.4.2	Trade-off: usabilità vs sicurezza	6
2.4.3	Aspetti etici e normativi	6
2.5	Inter-compatibilità	6
2.5.1	L'assenza di uno standard	7
2.5.2	Hub	7
2.5.3	Matter e thread	7
2.6	Progetti open-source	7
2.6.1	Vantaggi per la sicurezza	7
2.6.2	Confronto tra progetti	8
2.6.3	Home Assistant	8
2.6.4	Open Home Foundation	8

3	EspHome	11
3.1	Vantaggi e Applicazioni	11
3.2	Pipeline di Validazione, Traduzione e Compilazione	12
3.2.1	Validazione	12
3.2.2	Traduzione	12
3.2.3	Compilazione	13
3.3	Interconnessione e API	13
4	Il dispositivo	15
4.1	Caratteristiche principali	15
4.2	Dashi	16
4.2.1	Scocca	16
4.2.2	Hardware	18
4.2.2.1	Componenti elettronici	18
4.2.2.2	Schematico	20
4.2.2.3	Scheda elettronica personalizzata	22
4.2.2.4	Schematico generale	24
4.2.3	Software	24
4.2.3.1	Front-end	24
4.2.3.2	Back-end	36
5	conclusioni	45
5.1	Ricapitolazione	45
5.2	Considerazioni critiche	45
5.3	sviluppi futuri	46
	Bibliografia	49

Elenco delle figure

4.1	Modello 3D del dispositivo assemblato	17
4.2	Da sx a dx e dall'alto verso il basso: Placca frontale, placca posteriore, placca di ancoraggio e involucro per l'elettronica	18
4.3	Disegno circuitale componenti	20
4.4	Render 3d della PCB	22
4.5	Disegno circuitale della PCB	23
4.6	Disegni circuitali completi	24
4.7	Status bar	28
4.8	Navigation bar	28
4.9	Home Page	30
4.10	Pagina della stanza: Camera Mattia	33
4.11	Pagina di impostazioni	34
4.12	Albero di navigazione dell'interfaccia grafica	35
4.13	Stato dei pulsanti	43

Capitolo 1

Introduzione

Negli ultimi anni, il concetto di casa intelligente ha acquisito un ruolo sempre più centrale nella progettazione, ristrutturazione e manutenzione degli impianti elettrici residenziali. Questo crescente interesse è motivato da diversi fattori, tra cui il risparmio energetico, il miglioramento della sicurezza domestica, l'aumento del comfort abitativo e, soprattutto, la possibilità di interagire in maniera intuitiva con sistemi che diventano via via più sofisticati.

In questo contesto, uno degli elementi di maggiore rilevanza è rappresentato dall'accessibilità della domotica, sia in termini economici che tecnici. Storicamente, il settore era dominato da soluzioni closed-source, spesso costose e difficilmente personalizzabili. Tuttavia, l'avvento di piattaforme open-source ha radicalmente cambiato questo scenario.

Tra le piattaforme più influenti in questo ambito emerge Home Assistant, un sistema open-source supportato dalla Open Home Foundation, che si propone di unificare l'interazione tra dispositivi eterogenei attraverso un unico sistema centrale. Home Assistant consente l'integrazione di numerosi dispositivi e tecnologie differenti, favorendo un alto grado di interoperabilità, indipendentemente dal produttore o dal protocollo di comunicazione utilizzato.

All'interno di questo ecosistema si inserisce anche ESPHome, anch'esso sviluppato e mantenuto dalla Open Home Foundation. ESPHome è un framework che consente la creazione e la configurazione di dispositivi smart basati su microcontrollori, in particolare sugli ESP8266 e ESP32 prodotti da Espressif Systems. Uno dei principali punti di forza di ESPHome è la semplicità della programmazione, che avviene attraverso file di configurazione in formato YAML, rendendo la creazione di dispositivi intelligenti accessibile anche a chi non possiede competenze avanzate di sviluppo software.

I dispositivi realizzati con ESPHome possono essere facilmente integrati in Home Assistant, colmando così il vuoto lasciato dalle soluzioni commerciali troppo generiche o poco personalizzabili. In questo contesto si colloca il presente lavoro di tesi, il cui obiettivo è la progettazione e la realizzazione di un dispositivo domotico economico e completamente personalizzabile,

denominato “Dashi”.

Dashi è un dispositivo smart basato su microcontrollore ESP32, dotato di interfaccia grafica (HUI), sensori di temperatura e attuatori fisici, progettato per essere installato all’interno di una comune scatola da incasso 503. Il progetto si propone di fornire una soluzione compatta, discreta e facilmente installabile, che non richieda interventi strutturali sull’impianto esistente.

La sua versatilità consente di adattarlo a diversi scenari applicativi, tra cui il controllo di punti luce, la gestione di sensori ambientali, il monitoraggio degli ingressi e la supervisione di sistemi di sicurezza. Inoltre, la sua natura open-source e la compatibilità con ESPHome permettono a qualunque utente di modificarne il comportamento e l’interfaccia secondo le proprie esigenze specifiche.

Gli obiettivi specifici di questa tesi sono:

- Introdurre lo stato attuale della domotica
- Analizzare le potenzialità di ESPHome nel contesto della domotica fai-da-te;
- Definire e ottimizzare l’architettura hardware, includendo display, sensori e interruttori;
- Sviluppare l’interfaccia utente (HUI) per un controllo intuitivo e accessibile;

Capitolo 2

L'avvento della domotica

2.1 Definizione di domotica

Secondo la *Treccani*, la domotica è definita come:

«Branca dell'ingegneria che tratta dei sistemi di automazione per la gestione di edifici a uso abitativo e, più in generale, degli ambienti antropizzati. Una sua applicazione prevede l'integrazione tra alcune componenti di un edificio [...] e tecnologie dell'architettura, dell'automazione, dell'elettronica, dell'informatica, e delle telecomunicazioni.»[1]

2.2 Evoluzione storica

2.2.1 Il protocollo X10

Il primo sistema domotico commerciale risale al 1975, con lo sviluppo del protocollo X10 da parte di Pico Electronics. X10 introduceva un meccanismo di controllo remoto basato su:

- controllori e ricevitori installati all'interno degli impianti elettrici.
- indirizzi numerici (da 1 a 16) e codici di abitazione (da A a P).
- messaggi digitali trasmessi sulla linea di alimentazione, per attivare relè e dispositivi collegati[2][3].

Nonostante l'innovatività, X10 mostrò rapidamente limiti in termini di affidabilità del segnale e complessità di configurazione.

2.2.2 Barriere economiche e rischio di insuccesso

Nel corso degli anni Ottanta e Novanta, il predominio di soluzioni proprietarie a costi elevati frenò l'adozione di massa della domotica. Un contributo critico a questa percezione si deve all'articolo pubblicato nel 1992 dall'IEEE, che analizzò la sostenibilità economica del mercato di home automation. In tale studio si evidenziava come un termostato domotico potesse arrivare a costare fino a 12 000 \$, con stime di mercato comprese tra i 5 e i 10 miliardi di dollari per il decennio 1990–2000[4].

2.2.3 La rivoluzione wireless: IEEE 802.11

La svolta fu favorita dall'affermarsi dello standard IEEE 802.11, che introdusse la comunicazione tramite reti Wi-Fi e consentì di:

- eliminare il cablaggio puntuale di ogni singolo dispositivo, riducendo tempi e costi di installazione.
- incrementare la flessibilità negli interventi di retrofit, senza opere murarie invasive.
- abilitare l'interconnessione fra dispositivi eterogenei sfruttando infrastrutture di rete già diffuse.

Grazie alle soluzioni wireless si aprì la strada allo sviluppo di sistemi domotici economici, facili da integrare e personalizzabili, gettando le basi per la diffusione di piattaforme open-source quali Home Assistant ed ESPHome.

2.3 Scenario di mercato e tendenze

2.3.1 Previsioni di mercato

Negli Stati Uniti, le vendite di hardware per smart home sono attese crescere a un tasso annuale composto (CAGR) del 24,6% tra il 2015 e il 2019, raggiungendo 39 miliardi di dollari entro la fine del periodo [5]. Si prevede inoltre che verranno installati 45 milioni di dispositivi smart home entro il 2018, con un tasso di adozione dei termostati intelligenti pari al 43% nei successivi cinque anni.

2.3.2 Tendenze trainanti

Le principali tendenze che spingono il settore sono:

- **Stile di vita digitale:** crescente penetrazione di Internet, smartphone e tablet.
- **Internet of Things:** espansione di oggetti connessi, prevista in 300 miliardi di unità entro il 2020.
- **Cloud computing:** semplificazione dell'integrazione e gestione dei dati.
- **Robotica domestica:** aumento dei robot per le pulizie e altri compiti.
- **Sostenibilità ambientale:** tecnologie per la gestione efficiente dell'energia.

2.3.3 Principali investitori

Tra il 2012 e il 2013, le startup della smart home hanno attirato 468 milioni di dollari di venture capital[6] e il mercato ha visto una forte attività di fusioni e acquisizioni: ad esempio, Google–Nest (3,2 miliardi di \$) e Samsung–SmartThings.

2.3.4 Sfide principali

Le barriere per l'adozione di massa rimangono:

1. **Costo elevato** delle soluzioni complete.
2. **Sicurezza** e tutela dei dati personali.
3. **Flessibilità** limitata dall'interoperabilità tra marchi diversi.
4. **Gestibilità** percepita come complessa dagli utenti finali.

2.3.5 Aspetti non-market

La normativa è ancora frammentaria: si discute di privacy, minimizzazione dei dati e responsabilità in caso di violazioni. Organismi come la Federal Trade Commission hanno promosso workshop per definire linee guida e raccomandazioni [7].

2.4 Sicurezza

L'adozione di soluzioni per la smart home porta con sé evidenti criticità per quanto riguarda la sicurezza informatica, protezione della privacy e affidabilità dei dispositivi [8], [9].

2.4.1 Rischi specifici dei sistemi domotici

I sistemi domotici si basano spesso su hardware datato e codice non robusto, elementi che rischiano di compromettere la sicurezza. Tra le problematiche più ricorrenti vi sono:

- mancanza di supporto per l'autenticazione forte e multi-fattore.
- firmware vulnerabile a buffer overflow e privilege escalation.
- esposizione delle API di controllo sulla rete locale o pubblica.
- logging non cifrato delle attività utente su servizi remoti.

Un esempio emblematico è rappresentato dai numerosi casi di telecamere IP economiche che espongono flussi video non cifrati accessibili via browser [10]. Tali vulnerabilità sono spesso ignorate dai produttori, che puntano più alla velocità di distribuzione che alla sicurezza a lungo termine.

2.4.2 Trade-off: usabilità vs sicurezza

I sistemi commerciali, come Google Nest o Amazon Alexa, sono progettati per essere intuitivi e “plug-and-play”, ma spesso a scapito della trasparenza e della privacy [11], [12].

2.4.3 Aspetti etici e normativi

Il rispetto del Regolamento Generale sulla Protezione dei Dati (GDPR) impone obblighi specifici anche per i dispositivi IoT domestici:

- minimizzazione dei dati raccolti.
- possibilità per l'utente di accedere, rettificare e cancellare i propri dati.
- obbligo di trasparenza sulle finalità e sui metodi del trattamento.

2.5 Inter-compatibilità

Nello stato attuale esistono una moltitudine di protocolli per la domotica (Wifi, BT, Zigbee, ZWave, IR, KNX) e un altrettanto numero di protocolli proprietari sviluppati dalle singole aziende.

2.5.1 L'assenza di uno standard

Ogni azienda che sviluppa dispositivi domotici si affida a protocolli di comunicazioni differenti, addirittura creandone di nuovi. Questo comporta il totale isolamento verso tecnologie di comunicazioni differenti, creando un grande ostacolo per lo crescita della casa intelligente [13].

2.5.2 Hub

La soluzione proposta dalle grandi aziende che producono prodotti IoT converge quasi sempre sull'aggiunta dei cosiddetti hub, come SmartThings e Google Home, che hanno l'obiettivo di uniformare il più possibile, la giungla di applicazioni che annoiano l'utente finale [14], [15].

2.5.3 Matter e thread

Matter, precedentemente conosciuto come Project Connected Home over IP (CHIP), è uno standard di comunicazioni che favorisce l'interoperabilità tra diversi dispositivi IoT e sviluppato da Connectivity Standard Alliance, nata dalla cooperazione di Apple, Amazon, Google e Samsung [16]. Il nuovo standard utilizzerà la rete Wifi e in particolare la rete Thread, un nuovo protocollo rilasciato nel 2015, specificatamente progettato per la domotica ed è costituito da molti standard. Si pone dunque l'obiettivo di unificare le aziende che sfruttano sistemi proprietari sotto una piattaforma unica[17].

2.6 Progetti open-source

La necessità di indipendenza dal cloud dei colossi tech, la difficoltà di integrazione tra dispositivi e il desiderio di avere il controllo completo sulla propria casa, hanno portato allo sviluppo di progetti open-source.

2.6.1 Vantaggi per la sicurezza

Il paradigma open source si fonda sul principio della trasparenza. Questo permette, almeno in teoria, una maggiore sicurezza grazie alla possibilità di audit indipendenti e comunità che identificano e correggono le vulnerabilità [18]. Inoltre:

- le configurazioni sono spesso completamente personalizzabili.
- gli aggiornamenti possono essere gestiti localmente, riducendo la dipendenza da cloud esterni.
- gli utenti possono verificare esattamente quali dati vengono raccolti, come e quando.

2.6.2 Confronto tra progetti

Progetti come Home Assistant, supportato da una comunità globale, offrono strumenti integrati per la gestione locale della smart home, il tutto senza richiedere una connessione cloud. Analogamente, OpenHAB e Domoticz offrono framework modulari, compatibili con centinaia di dispositivi e protocolli, dal più semplice Zigbee fino a modelli proprietari retro-ingegnerizzati [19].

2.6.3 Home Assistant

Home Assistant è una piattaforma di domotica open source, scritta in Python, il cui sviluppo è stato avviato nel 2013 da Paulus Schoutsen. La piattaforma è progettata per essere eseguita interamente in locale, senza dipendenza da servizi cloud esterni, con l'obiettivo di restituire all'utente il pieno controllo sul proprio ambiente domestico digitale [20].

Home Assistant si distingue per un elevato grado di modularità e scalabilità, consentendo l'integrazione con oltre 2.000 dispositivi e servizi, tra cui standard diffusi come Zigbee, Z-Wave, MQTT, KNX, oltre a ecosistemi proprietari come Google Home e Amazon Alexa [21]. Grazie alla natura open-source del progetto, è possibile estendere la funzionalità della piattaforma attraverso lo sviluppo di componenti personalizzati, script avanzati e automazioni basate su logica condizionale complessa.

Un ulteriore valore aggiunto risiede nella forte comunità di sviluppatori e utenti che contribuisce attivamente al mantenimento del codice, alla redazione della documentazione e allo sviluppo di interfacce grafiche user-friendly [22]. Questo modello collaborativo rappresenta un esempio concreto di come il software open source possa offrire soluzioni robuste, sicure e sostenibili, spesso superiori a quelle commerciali in termini di flessibilità e adattabilità alle esigenze dell'utente.

2.6.4 Open Home Foundation

L'idea nasce nel 2013 quando il fondatore di Home Assistant, Paulus Schoutsen, scrive un piccolo programma in python per controllare le sue luci Philips Hue in locale. Negli anni a venire molti utenti hanno contribuito allo sviluppo di questo semplice script, facendolo diventare una piattaforma domotica in tutto e per tutto. Nel 2024 viene fondata la Open Home Foundation, una fondazione che raggruppa 240 progetti, standard e librerie di smart home, con l'obiettivo di salvaguardare la loro natura open-source [23].

Il loro motto, visibile nella loro pagina web recita:

«The Open Home Foundation fights for the fundamental principles of privacy, choice, and sustainability for smart homes. And for every person who lives in

one. It does this by supporting the development of open-source projects, and open connectivity and communication standards»

I principale obiettivi sono:

- **Promuovere standard aperti:** Favorisce l'adozione di tecnologie open-source e protocolli standard per garantire l'interoperabilità tra dispositivi di produttori diversi.
- **Supportare progetti open-source:** Sostiene attivamente progetti come ESPHome, Home Assistant, Zigbee2MQTT.
- **Tutela della privacy degli utenti:** Vuole realizzare una casa intelligente che funzioni senza obbligo di cloud esterni, mantenendo i dati all'interno della rete locale.
- **Educazione e accessibilità:** Fornisce documentazione, esempi e strumenti per rendere la domotica accessibile anche a non esperti.

Capitolo 3

EspHome

ESPHome è un framework open-source che permette di realizzare firmware personalizzati per microcontrollori basati su ESP8266 ed ESP32. Il suo scopo principale è quello di semplificare lo sviluppo di dispositivi Internet of Things (IoT), offrendo un'interfaccia di configurazione basata su file YAML. Gli utenti possono definire il comportamento dei dispositivi attraverso configurazioni testuali, che vengono poi compilate in codice C++ [24].

3.1 Vantaggi e Applicazioni

I principali vantaggi offerti da ESPHome includono:

- **Semplicità d'uso:** non è richiesta una conoscenza approfondita del linguaggio C++.
- **Flessibilità:** supporta una vasta gamma di sensori, attuatori e protocolli (I2C, SPI, UART, MQTT, ecc.).
- **Modularità:** è possibile estendere il sistema con componenti personalizzati.
- **Integrazione:** comunicazione nativa con Home Assistant via API o MQTT.
- **Controllo locale:** i dispositivi non necessitano di un cloud per il loro funzionamento.

Le applicazioni più comuni si focalizzano principalmente sulla versione obbistica di progetti di automazione domestica e monitoraggio ambientale, fino a prodotti industriali, supportati grazie all'etichetta *Made for EspHome* [25] come *Everything Presence One*: un dispositivo equipaggiato con sensori di movimento che permette di rilevare la presenza di entità nell'area circostante [26].

3.2 Pipeline di Validazione, Traduzione e Compilazione

EspHome utilizza due linguaggi principali:

- **C++**: Eseguito sui microcontrollori per comunicare con EspHome e interfacciarsi con i sensori.
- **YAML**: Utilizzato per definire la configurazione del dispositivo e facilitare la scrittura dell'utente.

Definito il codice per Dashi, reperibile al seguente link e analizzato nella sezione Hardware 4.2.3, deve seguire tre fasi distinte: validazione, traduzione e compilazione, dalle quali si ottiene il firmware da caricare sul dispositivo [27], [28].

3.2.1 Validazione

La prima fase si occupa di validare la configurazione YAML redatta dall'utente. Attraverso lo script Python *config_validation.py*, che fa uso della libreria Voluptuous [29], si verifica che i campi obbligatori siano presenti, che i campi opzionali non presenti abbiano la configurazione di default e che tutti i tipi di dato siano corretti (ad esempio: stringhe, interi, ...).

Per effettuare queste operazioni, tutti i moduli configurabili dall'utente devono definire una variabile speciale `CONFIG_SCHEMA`, di seguito un esempio:

```
1 import esphome.config_validation as cv
2
3 CONF_MY_REQUIRED_KEY = 'my_required_key'
4 CONF_MY_OPTIONAL_KEY = 'my_optional_key'
5
6 CONFIG_SCHEMA = cv.Schema({
7     cv.Required(CONF_MY_REQUIRED_KEY): cv.string,
8     cv.Optional(CONF_MY_OPTIONAL_KEY, default=10): cv.int_,
9 }).extend(cv.COMPONENT_SCHEMA)
```

Verificato che la configurazione risulta corretta, questa può passare alla fase di traduzione.

3.2.2 Traduzione

La fase di traduzione in codice permette di "convertire" la configurazione YAML in codice C++ utilizzando lo script Python *cpp_generator.py*.

Ad esempio, la configurazione del display vista nel blocco di codice 4.2.3.2 viene "tradotta" nella seguente struttura:

```
1 auto display_1 = new ILI9XXXDisplay();
2 display_1->set_model(ILI9XXX_MODEL_ST7796);
3 display_1->set_spi_id("lcd");
4 display_1->set_dc_pin(GPI013);
```

```
5 display_1->set_reset_pin(GPIO20);
6 display_1->set_cs_pin(GPIO14);
7 display_1->set_invert_colors(true);
8 display_1->set_update_interval(5000);
9 display_1->set_auto_clear_enabled(false);
10 display_1->set_show_test_card(false);
11 display_1->set_width(480);
12 display_1->set_height(320);
13
14 display_1->set_swap_xy(true);
15 display_1->set_mirror_x(true);
16 display_1->set_mirror_y(true);
17
18 App.register_component(display_1);
19 App.register_display(display_1);
```

Effettuate le operazioni di conversione della configurazione YAML, questa può passare alla fase finale di compilazione.

3.2.3 Compilazione

Il processo di generazione del firmware viene affidato a PlatformIO, un ecosistema di sviluppo open-source progettato per la programmazione di dispositivi embedded, in particolare microcontrollori come ESP32, ESP8266, Arduino, STM32, e molti altri. Si tratta di un ambiente che fornisce strumenti per la compilazione, il debugging, la gestione delle librerie e il caricamento del firmware su dispositivi hardware [30].

In definitiva si ottiene un file chiamato `firmware.bin`, che può essere successivamente caricato sul dispositivo attraverso una interfaccia USB o attraverso un aggiornamento OTA.

3.3 Interconnessione e API

ESPHome non è solo uno strumento per generare firmware per microcontrollori: è anche pensato per integrarsi con sistemi domotici esterni attraverso API [31]. Grazie a queste interfacce, i dispositivi programmati con ESPHome possono interagire in tempo reale con altre piattaforme, inviare dati, ricevere comandi o essere controllati da remoto.

Le principali API supportate da ESPHome includono:

- **MQTT**: Message Queuing Telemetry Transport è un protocollo molto usato nella comunicazione tra dispositivi IoT, leggero ed efficiente. Il suo funzionamento si basa sull'architettura *Publish/Subscribe*, dove ogni dispositivo può inviare e raccogliere dati su un *topic*, sfruttando un server centrale detto *Broker* [32].

- **Richieste HTTP:** ESPHome può essere configurato per inviare richieste HTTP GET o POST verso server esterni. Essendo questo protocollo molto comune, può essere facilmente integrato in qualsiasi sistema [33].
- **Home Assistant:** L'integrazione nativa con Home Assistant è una delle caratteristiche chiave di ESPHome, si basa sul sistema *Protocol Buffers* che permette di serializzare i messaggi in modo compatto [34], [35].

Un esempio di configurazione viene riportato nel blocco di codice 4.2.3.2, la cui sicurezza viene garantita da una chiave codificata in Base64 da 32 byte[36].

Se ciò non fosse sufficiente o se grandi aziende volessero integrare EspHome all'interno dei loro sistemi, è possibile scrivere moduli personalizzati per comunicare con qualsiasi protocollo.

Capitolo 4

Il dispositivo

Grazie alla flessibilità di EspHome, facilmente configurabile utilizzando codice di configurazione YAML e alla inter-compatibilità di Home Assistant, che integra al suo interno i dispositivi domotici, nasce Dashi

4.1 Caratteristiche principali

Le caratteristiche principali di questo dispositivo sono:

- **Inter-operabilità:** grazie all'integrazione tra EspHome e Home assistant, il dispositivo permette di controllare tutti i dispositivi presenti nella propria abitazione, tra cui sensori e attuatori
- **Interfaccia flessibile e curata:** grazie alla semplicità di configurazione di EspHome e all'utilizzo della libreria grafica LVGL è possibile realizzare un'interfaccia moderna e altamente personalizzabile, che si adatta al meglio alle necessità dell'utente
- **Facilità di installazione:** Il dispositivo è facilmente installabile in una comune scatoletta da muro 503 e viene alimentato direttamente a 230VAC, non è necessario effettuare alcuna opera muraria e può rimpiazzare i classici attuatori
- **Indipendenza da cloud esterni:** il dispositivo comunica tramite connessione Wifi con il Server di Home Assistant, senza passare per cloud esterni, favorendo così la sicurezza dei dati.

4.2 Dashi

Il nome Dashi nasce dall'unione di due concetti chiave che riflettono l'essenza del dispositivo. Da un lato, richiama il dashi, un brodo tradizionale della cucina giapponese, elemento base e fondante di molte preparazioni: un riferimento simbolico alla funzione centrale del dispositivo all'interno dell'ambiente domotico [37]. Dall'altro lato, il nome richiama la parola inglese dashboard, ovvero pannello di controllo, suggerendo in modo diretto la funzione principale del dispositivo: visualizzare e gestire in modo intuitivo i dispositivi domotici della casa. Il risultato è un nome evocativo, semplice e riconoscibile, che fonde cultura, tecnologia e funzionalità.

4.2.1 Scocca

La struttura esterna del dispositivo è stata progettata utilizzando software CAD (Computer-Aided Design), tenendo conto di ergonomia, estetica e funzionalità. La scocca è stata realizzata mediante stampa 3D con tecnologia SLA (Stereolitografia), utilizzando materiale plastico ABS-like per garantire un buon compromesso tra rigidità, resistenza e facilità di stampa.

Il design modulare della scocca permette una facile apertura per interventi di manutenzione o aggiornamenti hardware, e include appositi fori per il montaggio a parete e si compone di due elementi principali che si accoppiano magneticamente: la piastra posteriore che si ancora alla scatola 503 e la scocca che contiene i componenti.

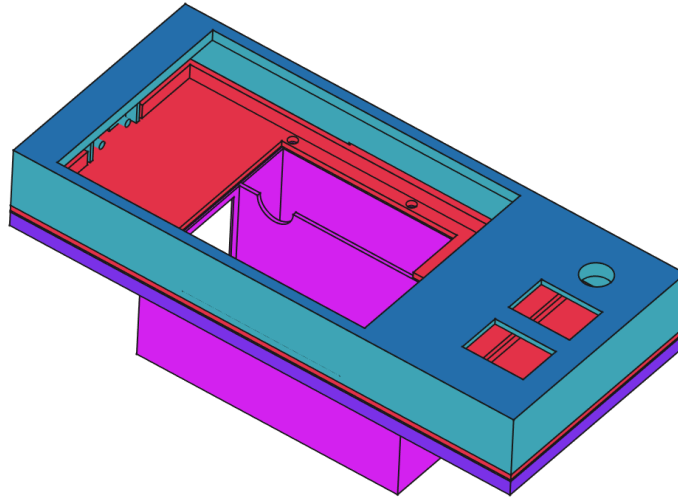


Figura 4.1: Modello 3D del dispositivo assemblato

La scocca si compone di tre elementi principali:

- **Placca frontale:** elemento che contiene il display, il sensore ambientale, l'encoder rotativo. Si incastra al blocco inferiore utilizzando degli innesti a scatto.
- **Placca posteriore + involucro per elettronica:** elemento principale della struttura che contiene la parte elettronica. Sul retro sono posizionati dei magneti per rimuoverla dalla placca di ancoraggio.
- **Placca di ancoraggio:** elemento che si intrappone tra il blocco posteriore e la scatola 503 che permette una facile installazione e manutenzione.

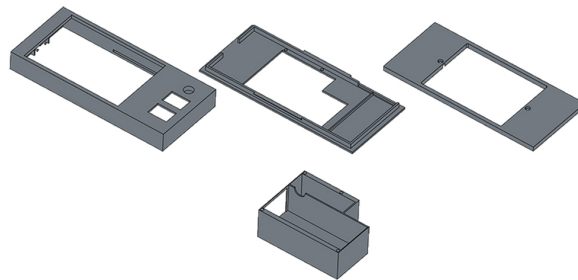


Figura 4.2: Da sx a dx e dall'alto verso il basso: Placca frontale, placca posteriore, placca di ancoraggio e involucro per l'elettronica

4.2.2 Hardware

4.2.2.1 Componenti elettronici

All'interno della scocca sono stati installati una selezione di componenti integrati

- **Microcontrollore ESP32-S3 N16R8:** Modulo della famiglia di ESP32 prodotto da Espressif, campione di rapporto qualità prezzo e dotato delle seguenti caratteristiche [38]:
 - **Xtensa® dual-core 32-bit LX7:** microprocessore che raggiunge frequenze di 240MHz.
 - **FLASH 16MB, PSRAM 8MB:** memoria FLASH che permette di scrivere al suo interno una quantità di codice elevata e memoria PSRAM che concilia l'alta capacità di una DRAM con la semplicità della SRAM, che permettono al motore grafico di renderizzare le interfacce senza alcun lag.
 - **Antenna Wifi e Bluetooth:** permettono al dispositivo di comunicare tramite onde radio, senza la necessità di aggiungere moduli e/o antenne esterne.
 - **Periferiche:** 36 GPIOs, con interfacce I2C e SPI favoriscono l'integrazione di sensori e attuatori.
- **Display IPS:** schermo touch da 3.5 pollici:[39]:
 - **Risoluzione 320x480:** perfetta per realizzare una interfaccia ricca di dettagli.
 - **Connettività:** cavo FPC da 14 pin per comunicare con il microcontrollore.
 - **Driver Display:** il display è controllato dal driver ST7796U con interfaccia SPI.

- **Driver Touchscreen:** la gestione del touchscreen è affidata al driver FT6336U con interfaccia I2C.
 - **Dimensioni 55.5(W)x98.0(H)x14.28(D):** contenuto nella spazio permette di mantenere minime le dimensioni della scocca.
- **Rotary Encoder KY-040:** encoder rotativo con pulsante [40].
- **Sensore ambientale BME280:** montato su breakout board con interfaccia I2C permette di controllare temperatura, umidità e pressione dell'ambiente [41].
- **Resistori:** Resistori assiali di varie caratteristiche utilizzati per pull-up di bottoni e interfacce I2C.
- **Pulsanti:** microinterruttori 4 poli per controllare il display.
- **Alimentatore:** un piccolo convertitore da 230VAC a 5VDC permette di alimentare il dispositivo in modo pratico.

Tutti i sensori (ad esclusione del display che utilizza un connettore FPC) sono stati terminati utilizzando connettori JST, per una facile installazione e manutenzione

4.2.2.2 Schematico

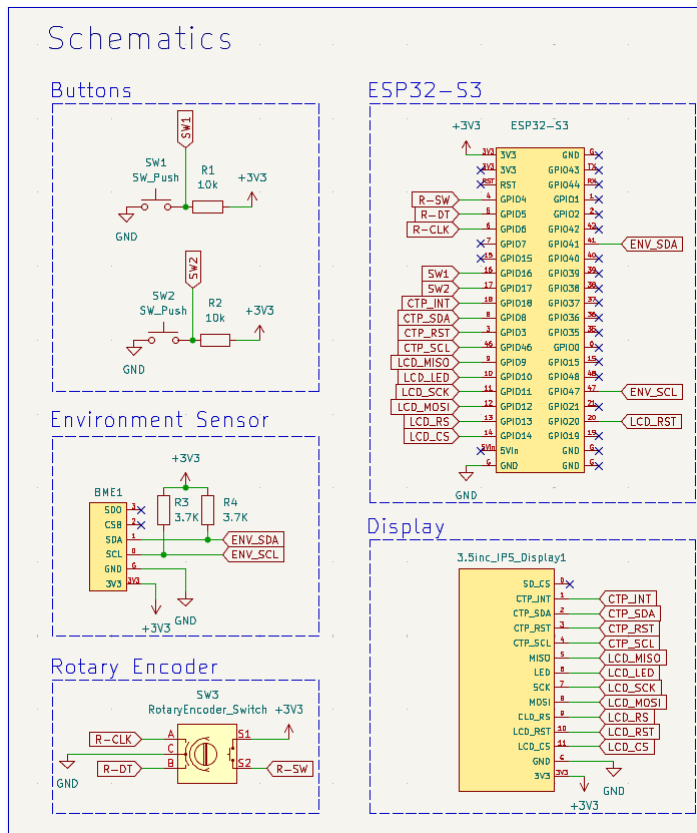


Figura 4.3: Disegno circuitale componenti

Lo schematico, realizzato con KiCad [42], sintetizza la disposizione dei componenti sopra esposti e facilita la comprensione del circuito stesso, si analizza di seguito la sua struttura:

- **Pulsanti:** nel circuito vengono utilizzati due pulsanti, nella configurazione con resistori di pull-up da $10k\Omega$, la cui uscita è collegata alla scheda principale.
- **Sensore Ambientale:** riporta pressione, temperatura e umidità utilizzando l'interfaccia I2C, sono presenti resistori di pull-up come da specifiche [41].
- **Encoder rotativo:** riporta il senso di rotazione attraverso R-CLK e R-DT, integra anche un pulsante, direttamente collegato alla scheda principale.
- **Display:** principale elemento della scheda si divide in due componenti: LCD e pannello touch
 - **Pannello touch:** interfaccia I2C per comunicazione con il pannello touch
 - * **CTP_INT:** pin di interrupt, segnala al microcontrollore che è avvenuto un evento touch.

- * **CTP_SDA**: linea dati del bus I2C.
- * **CTP_RST**: pin di reset del touch controller.
- * **CTP_SCL**: linea di clock del bus I2C.
- **LCD**: comunica con interfaccia SPI
 - * **LCD_MISO**: Master In Slave Out, usato per ricevere dati dal display (raramente usato nei display LCD base).
 - * **LCD_LED**: retroilluminazione del display.
 - * **LCD_SCK**: clock SPI per la sincronizzazione della comunicazione.
 - * **LCD_MOSI**: Master Out Slave In, invia dati al display.
 - * **LCD_RS**: Register Select, seleziona tra istruzioni o dati da inviare.
 - * **LCD_RST**: reset hardware del display.
 - * **LCD_CS**: Chip Select, abilita la comunicazione con il display.
- **ESP32-S3**: la scheda principale si occupa di controllare i sensori ad essa connessi, l'assegnazione dei pin risulta:

- **Pulsanti**

- * SW1 → GPIO16
- * SW2 → GPIO17

- **Sensore ambientale**

- * ENV_SDA → GPIO41
- * ENV_SCL → GPIO47

- **Encoder rotativo**

- * R_SW → GPIO4
- * R_DT → GPIO5
- * R_CLK → GPIO6

- **Display**

- * LCD_MISO → GPIO9
- * LCD_LED → GPIO10
- * LCD_SCK → GPIO11
- * LCD_MOSI → GPIO12
- * LCD_RS → GPIO13
- * LCD_CS → GPIO14

- * LCD_RST → GPIO20
- * CTP_SDA → GPIO8
- * CTP_SCL → GPIO46
- * CTP_INT → GPIO18
- * CTP_RST → GPIO3

4.2.2.3 Scheda elettronica personalizzata

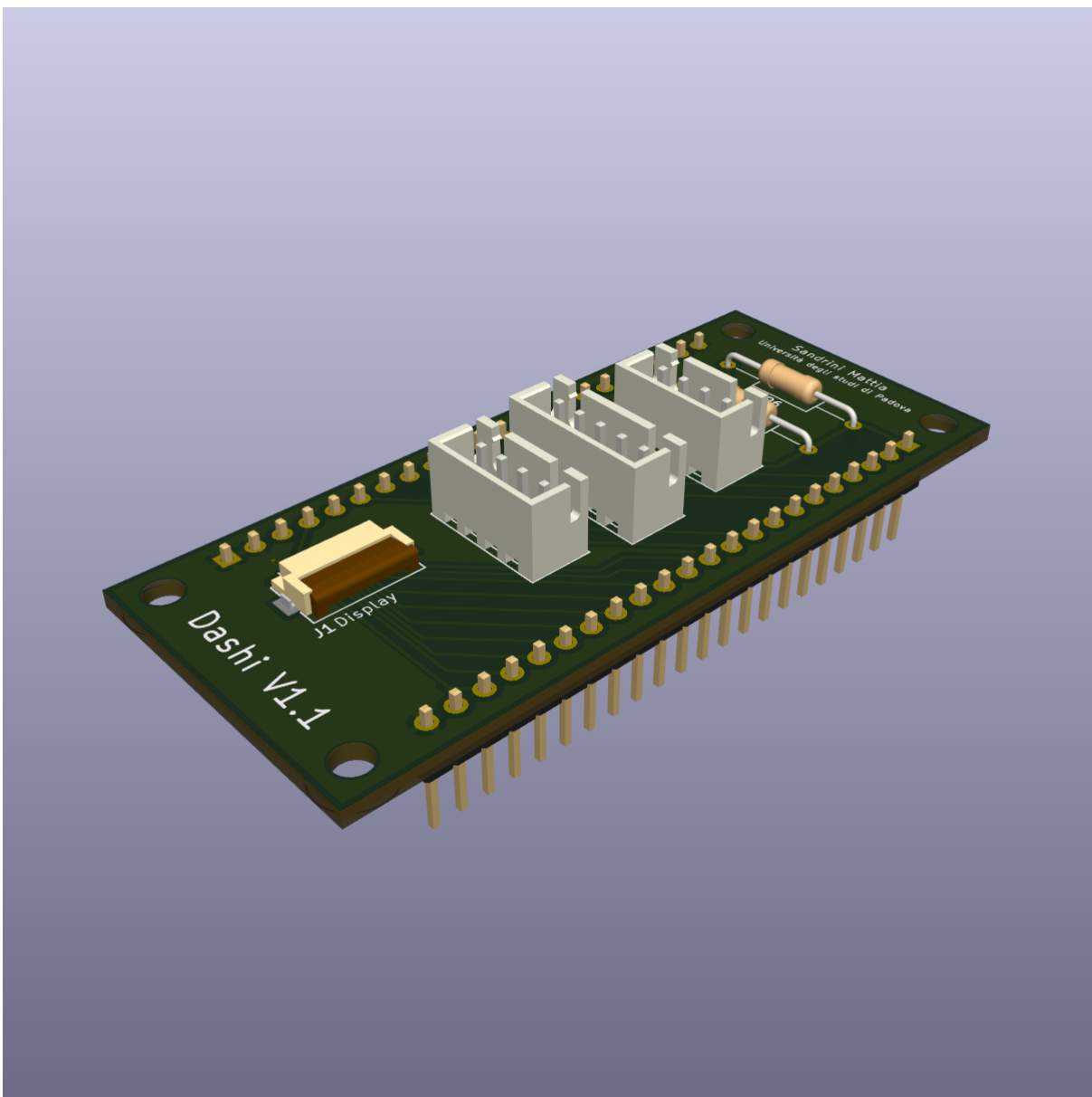


Figura 4.4: Render 3d della PCB

Data la presenza di sensori, attuatori e display da racchiudere in poco spazio si è reso necessario la realizzazione di una scheda elettronica per interfacciare in modo efficace i componenti al microcontrollore e si compone principalmente di due parti:

1. **Parte superiore:** riporta i connettori JST-XH con pitch di 2.5mm, rispettivamente due da 4 pin (per pulsanti e sensori ambientale) [43] e uno da 5 pin (per encoder rotativo) [44], un connettore FPC-05F-14P [45] da 14 pin (per display), due resistori di pull-up per i pulsanti e due header da 22 pin con passo da 2.54mm
2. **Parte inferiore:** riporta l'altra estremità dei due header, ai quali verrà saldata la scheda ESP32-S3

Qui sotto è riportato lo schematico della scheda, molto simile a quanto visto sopra ma riadattato alle necessità di progettazione di una scheda elettronica.

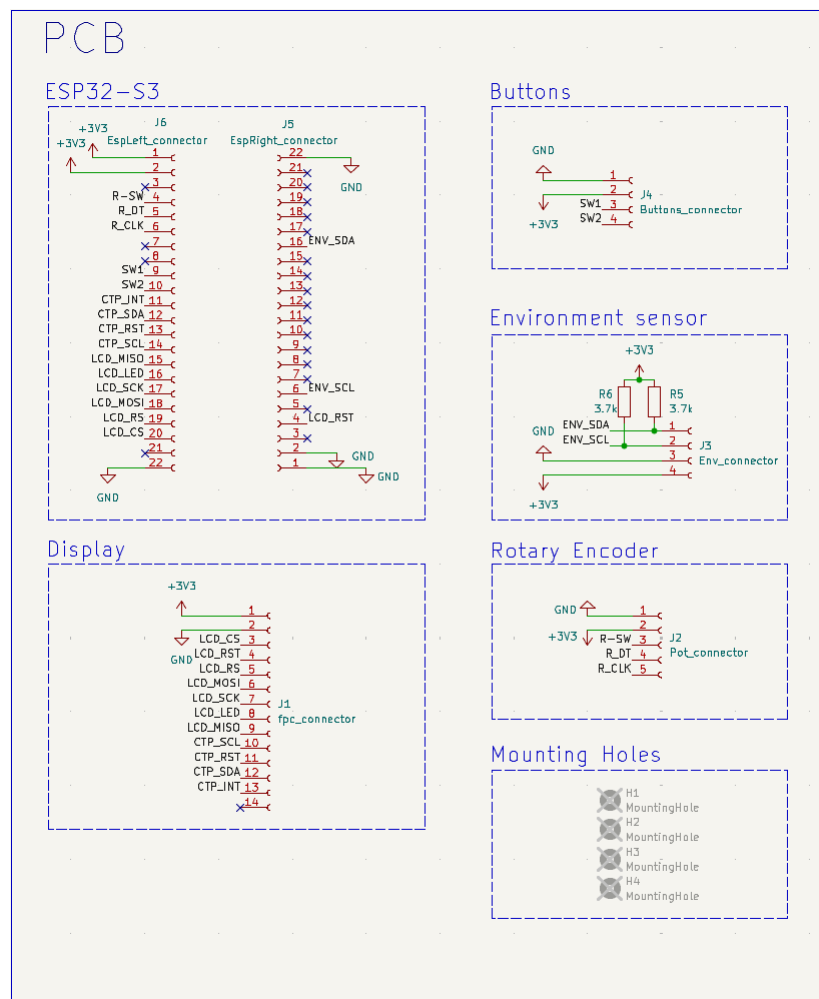


Figura 4.5: Disegno circuitale della PCB

4.2.2.4 Schematico generale

Viene riportato per semplicità di lettura lo schema del circuito in tutte le sue parti.

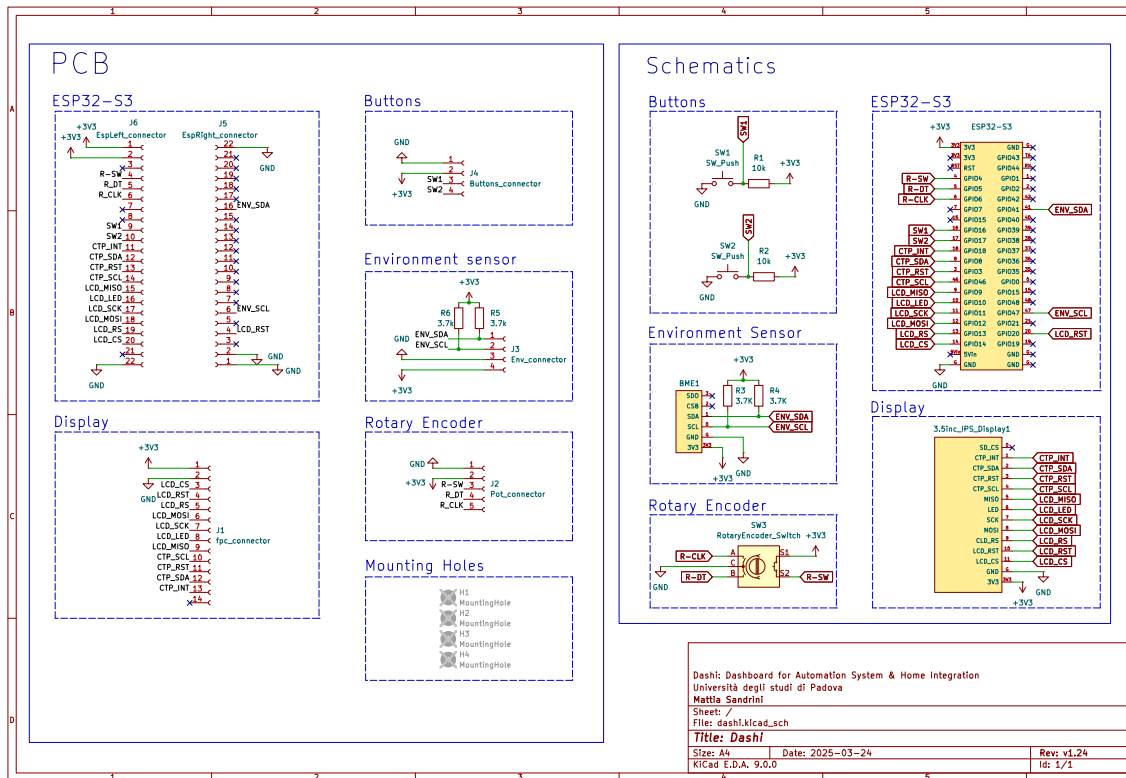


Figura 4.6: Disegni circuitali completi

4.2.3 Software

Il software della scheda, meglio definito come configurazione YAML di EspHome con aggiunta di funzioni LAMBDA, si occupa di interfacciare l'utente al dispositivo (front-end) e successivamente interfaccia Dashi al server Home Assistant (back-end).

4.2.3.1 Front-end

L'interfaccia grafica si basa sulla libreria open-source per microcontrollori LVGL (Light and Versatile Graphics Library) [46], sponsorizzata da grandi aziende come Xiaomi, Texas Instruments, ARM, Espressif e molte altre. E' perfettamente integrata in EspHome [47]. La libreria si compone di tre livelli:

- **Driver di basso livello:** si occupa dell'interazione dell'hardware come display, touchscreen e econdor rotativi.

- **LVGL core:** cuore pulsante della libreria che si occupa della gestione degli eventi e del rendering dei widget.
- **Layer utente:** definisce l'interfaccia utente

La struttura grafica si basa su widget: elementi predefiniti le cui proprietà di stile (colore, posizione, dimensione, ...) possono essere gestite in maniera dinamica, di seguito una raccolta di quelli più utilizzati da Dashi

- **Object:** è la struttura fondamentale della libreria LVGL. Rappresenta un oggetto grafico generico, e costituisce la base di tutti i widget (pulsanti, etichette, slider, ecc.). Tutti gli altri oggetti ne ereditano caratteristiche come dimensione, posizione, stili e sistema di eventi
- **Buttons:** è uno dei componenti base dell'interfaccia. È un contenitore che può contenere altri oggetti e supporta eventi di pressione e rilascio.
- **Buttonmatrix:** rappresenta una griglia di pulsanti, utile per tastiere virtuali, calcolatrici, o menu compatti.
- **Label:** utilizzata per visualizzare testo statico o dinamico. È molto leggera in termini di risorse ed essenziale per mostrare messaggi, descrizioni o valori.
- **Spinbox:** consente all'utente di modificare un valore numerico mediante passi, utile per impostazioni di parametri come timer, temperatura o velocità.
- **Slider:** rappresenta un controllo scorrevole per selezionare un valore all'interno di un range continuo. È intuitivo e utile per regolare luminosità, volume, velocità.
- **Dropdown:** è un componente che permette di selezionare un valore da un elenco a discesa, ottimale per menu compatti e impostazioni con opzioni discrete.

Si analizzano ora le parti salienti del codice che permettono di realizzare l'interfaccia grafica, partendo da una visione generale della struttura per addentarsi nelle specifiche sotto-chiavi. **DISCLAIMER:** alcune chiavi, come quelle di definizione di stile o quelle superflue al contesto di analisi, saranno omesse per ottimizzare lo spazio e facilitare la lettura del codice. Il codice completo può essere reperito al seguente link.

```
1 lvgl:  
2   encoders:  
3   ...  
4   on_idle:  
5   ...  
6   theme:
```

```

7  ...
8  top_layer:
9  ...
10 pages:
11 ...

```

La chiave `lvgl` permette di importare e utilizzare la libreria grafica:

- **Chiave `encoders`:** permette di utilizzare un encoder rotativo per interagire con l'interfaccia.

```

1  encoders:
2      sensor: rot_encoder
3      enter_button: rot_button
4      group: rot_control
5

```

- **`sensor: rot_encoder`:** associa il sensore *rot_encoder* all'encoder per controllare l'interfaccia grafica.
- **`enter_button : rot_button`:** associa il sensore *rot_button* all'encoder per controllare l'interfaccia grafica.
- **`group: rot_control`:** crea il gruppo *rot_control*, che sarà poi assegnati ai widget che potranno interagire con l'encoder.

- **chiave `on_idle`:** permette di eseguire determinate azioni quando `lvgl` si trova nello stato *on_idle*, ovvero quando non c'è interazione con lo schermo.

```

1  timeout: !lambda "return (120 * 1000);"
2  then:
3      - logger.log: "LVGL is idle"
4      - light.turn_off: back_light
5      - lvgl.pause:
6

```

Ovvero: dopo un tempo di *120 secondi*, spegni lo schermo e metti in pausa il render degli widget

- **Chiave `theme`:** permette di definire lo stile degli widget

```

1  theme:
2  # set all your labels to use your custom defined font
3  button:
4      bg_color: 0x2F8CD8
5      bg_grad_color: 0x005782
6      bg_grad_dir: VER
7      bg_opa: COVER
8      border_color: 0x0077b3
9      border_width: 1
10     text_color: 0xFFFFFFFF

```

```

11     pressed: # set some button colors to be different in pressed state
12         bg_color: 0x006699
13         bg_grad_color: 0x00334d
14     checked: # set some button colors to be different in checked state
15         bg_color: 0x1d5f96
16         bg_grad_color: 0x03324A
17         text_color: 0xffff300
18     buttonmatrix:
19         ...
20     switch:
21         ...
22     slider:
23         ...
24

```

Analizzando per esempio la chiave *button*, le sottochiavi si occupano della definizione del colore dello sfondo (con gradiente), colore del testo e della definizione del colore del pulsante quando questo si trova nello stato premuto (sottochiave *pressed*) o attivo (sottochiave *checked*). Viene effettuato lo stesso anche per gli altri widget (buttonmatrix, switch, slider)

- **Chiave top_layer:** si occupa della definizione del layer che sovrasta tutti gli altri, utilizzato qui per creare una barra di stato nella zona alta e una barra di navigazione nella zona bassa.

```

1     top_layer:
2     widgets:
3         - obj:
4             ...
5             widgets:
6                 - label:
7                     id: greetings
8                     text: "Caricamento..."
9                     ...
10                - label: # show home assistant connection status
11                    text: "\U000F048B"
12                    id: lbl_hastatus
13                    ...
14                - label: # show wifi connection status
15                    text: "\uF1EB"
16                    id: wifi_status
17                    hidden: true
18                    ...
19                - label: # show date and time
20                    text: "-----"
21                    id: datetime
22                    hidden: false
23                    ...
24            - buttonmatrix: #navigation bar
25                ...
26            items:
27                styles: header_footer

```

```

28     rows:
29         - buttons:
30             - id: page_prev
31               width: 140
32               text: "\uF053"
33               on_press:
34                   then:
35                       lvgl.page.previous:
36             - id: page_home
37               width: 140
38               text: "\uF015"
39               on_press:
40                   then:
41                       lvgl.page.show: main_page
42             - id: page_next
43               width: 140
44               text: "\uF054"
45               on_press:
46                   then:
47                       lvgl.page.next:
48             - id: page_settings
49               width: 5
50               text: "\uF013"
51               on_press:
52                   then:
53                       lvgl.page.show: settings_page
54

```

si compone di due widget

- **obj**: racchiude la status bar superiore, dove possiamo trovare: lo stato di connessione con il server Home assistant (*id: lbl_hastatus*, lo stato di connessione wifi (*id: wifi_status*), la data con orario (*id: datetime*) e un messaggio di saluto personalizzato (*id: greetings*)



Figura 4.7: Status bar

- **buttonmatrix**: definisce la nav bar inferiore composta da 4 elementi: pulsante pagina avanti, pulsante pagina indietro, pulsante homepage e pulsante impostazioni, che come specificato dalla sottochiave *on_press*, quando vengono cliccati mostrano la pagina specificata (*lvgl.page.show: ***_page*)



Figura 4.8: Navigation bar

- **Chiave pages**: definisce le pagine che compongono l'interfaccia

```

1  pages:
2      - id: main_page
3        ...
4      - id: bedroom_mattia_page
5        ...
6      - id: bedroom_federico_page
7        ...
8      - id: bedroom_parents_page
9        ...
10     - id: livingroom_page
11       ...
12     - id: corridor_page
13       ...
14     - id: garden_page
15       ...
16     - id: kitchen_page
17       ...
18     - id: office_page
19       ...
20     - id: serverroom_page
21       ...
22     - id: settings_page
23       ...
24

```

L'interfaccia è quindi composta da 11 pagine, di cui 9 rappresentano delle stanze di una abitazione, si analizzano nello specifico alcune pagine caratterizzate da elementi strutturali e di novità.

- **Pagina Principale (*main_page*):** Questa pagina viene mostrata all'avvio del sistema e restituisce una interfaccia sintetica dello stato dell'abitazione e permette di raggiungere in modo rapido tutte le pagine

```

1  - id: main_page
2      scrollable: false
3      widgets:
4          - obj:
5              ...
6              id: weather_buttons
7              layout:
8                  type: GRID
9              ...
10             widgets:
11                 - label:
12                     text: "\U000F14E4"
13                     id: weather_forecast_condition_icon
14                     ...
15                 - label:
16                     text: "Sconosciuto"
17                     id: weather_forecast_condition_name
18                     ...
19                 - label:
20                     text: "Temp. est.:"

```

```

21         ...
22     - ...
23     - obj:
24         ...
25         id: main_page_buttons
26         layout:
27             type: FLEX
28             ...
29         widgets:
30             - button:
31                 id: but_bedroom_mattia
32                 group: rot_control
33                 ...
34                 on_press:
35                     then:
36                         - lvgl.page.show: bedroom_mattia_page
37             - button:
38                 id: but_bedroom_federico
39                 ...
40             - button:
41                 id: but_bedroom_parents
42                 ...
43             - ...
44 
```

Si compone di due widget:

- * **id: *weather_buttons(obj)***: utilizzato per creare una scheda che presenta le condizioni meteo attuali, fa uso del layout GRID (*riga 8*), per disporre gli elementi in modo ordinato.
- * **id: *main_page_buttons(obj)***: utilizzato per creare una scheda contenente nove bottoni che permettono di navigare velocemente tra le altre pagine, fa uso del layout FLEX (*riga 50*) per disporre gli elementi. La navigazione avviene grazie alla sotto-chiave `on_press` che fa ausilio dell'azione `lvgl.page.show: ***_page`



Figura 4.9: Home Page

– Camera Mattia (*bedroom_mattia_page*)

```

1  - id: bedroom_mattia_page
2      widgets:
3      - obj:
4          ...
5          id: bedroom_mattia_welcome
6      widgets:
7      - label:
8          text: "Camera Mattia"
9          ...
10     - label:
11         id: lbl_bedroom_mattia_temperature
12         text: "-- °C"
13         ...
14     - obj:
15         ...
16         id: bedroom_mattia_buttons
17     widgets:
18     - label:
19         text: "Luce principale"
20     - button:
21         ...
22         id: btn_bedroom_mattia_light_main
23     widgets:
24     - label:
25         id: lbl_bedroom_mattia_light_main
26         text: "\U000F0336" # mdi-lightbulb-outline
27         ...
28     on_click:
29         - homeassistant.action:
30             action: light.toggle
31             data:
32                 entity_id: light.luci_principali
33     - obj: #spacer
34         ...
35     - label:
36         text: "Luce letto"
37     - button:
38         ...
39         id: btn_bedroom_mattia_light_secondary
40     widgets:
41     - label:
42         id: lbl_bedroom_mattia_light_secondary
43         text: "\U000F17F5"
44         ...
45     on_press:
46         - homeassistant.action:
47             action: light.toggle
48             data:
49                 entity_id: light.luci_letto
50     - obj: #spacer
51         ...
52     - label:
53         text: "Condizionatore"
54     - obj:

```

```

55     ...
56     widgets:
57         - button:
58             id: spin_down
59             on_click:
60                 - lvgl.spinbox.decrement: sb_bedroom_mattia_thermostat
61             widgets:
62                 - label:
63                     text: "-"
64         - spinbox:
65             id: sb_bedroom_mattia_thermostat
66             ...
67             on_value:
68                 then:
69                     - homeassistant.action:
70                         action: climate.set_temperature
71                         data:
72                             temperature: !lambda return x;
73                             entity_id: climate.condizionatore
74         - button:
75             id: spin_up
76             on_click:
77                 - lvgl.spinbox.increment: sb_bedroom_mattia_thermostat
78             widgets:
79                 - label:
80                     text: "+"
81         - button:
82             id: btn_bedroom_mattia_thermostat_state
83             ...
84             widgets:
85                 - label:
86                     id: lbl_bedroom_mattia_thermostat_state
87                     text: "Sconosciuto"
88                 ...
89             on_click:
90                 - homeassistant.action:
91                     action: climate.toggle
92                     data:
93                         entity_id: climate.condizionatore
94

```

La pagina si compone di due widget:

- * **id: bedroom_mattia_welcome(obj)**: Posto nella parte superiore contiene il titolo della stanza e le informazioni relative alla temperatura di quest'ultima
- * **id: bedroom_mattia_buttons(obj)**: contiene a sua volta pulsanti per controllare luci e condizionatore:
 - **id: btn_bedroom_mattia_light_main, btn_bedroom_mattia_light_secondary**: fanno uso della sottochive *on_click* per chiamare l'azione *homeassistant.action*, che necessita delle chiavi *action* (definisce l'azione da compiere) e *data* (definisce il soggetto dell'azione)

- **id: sb_bedroom_mattia_thermostat**: spinbox che permette di inserire la temperatura desiderata, altrimenti modificabile utilizzando i pulsanti con id: *spin_down* e *spin_up*. Tale temperatura viene trasmessa utilizzando sempre l'azione *homeassistant.action*, ma con chiave *action* differente, trattandosi questa volta di una entità di tipo *climate*



Figura 4.10: Pagina della stanza: Camera Mattia

– Impostazioni (*settings_page*):

```

1  - id: settings_page
2      skip: true
3      widgets:
4          - obj:
5              ...
6              widgets:
7                  - label:
8                      text: "Impostazioni"
9                      ...
10         - obj:
11             ...
12             widgets:
13                 - label:
14                     ...
15                     text: "Luminosità"
16                 - slider:
17                     ...
18                     id: screen_brightness_slider
19                     on_value:
20                         - light.control:
21                             id: back_light
22                             brightness: !lambda "return (x / 100.0f);"
23                         - globals.set:
24                             id: lcd_brightness
25                             value: !lambda return x;
26                 - label:
27                     ...
28                     text: "Formato orario"
29                 - dropdown:

```

```

30     ...
31     id: dropdown_id
32     options:
33         - 0-12
34         - 0-24
35     selected_index: 1
36     on_value:
37         - globals.set:
38             id: time_format
39             value: !lambda return x;
40         - lambda: id(time_update).execute();
41 - label:
42     ...
43     id: lbl_wifi_status
44     text: "Sconosciuto"
45 - label:
46     ...
47     id: lbl_hass_status
48     text: "Sconosciuto"
49 - label:
50     ...
51     id: lbl_esphome_version
52     text: "Versione sconosciuta"
53 - button:
54     ...
55     widgets:
56         - label:
57             text: "Riavvio del sistema"
58     on_press:
59         - button.press: esp_reboot
60

```

La pagina impostazioni permette di gestire i parametri dell'interfaccia, come luminosità (utilizzando un *slider*, *id: screen_brightness_slider*), formato orario (utilizzando un *dropdown menu*, *dropdown_id* e fornisce un pulsante per il riavvio del sistema



Figura 4.11: Pagina di impostazioni

Lo schema di navigazione completo viene di seguito riportato

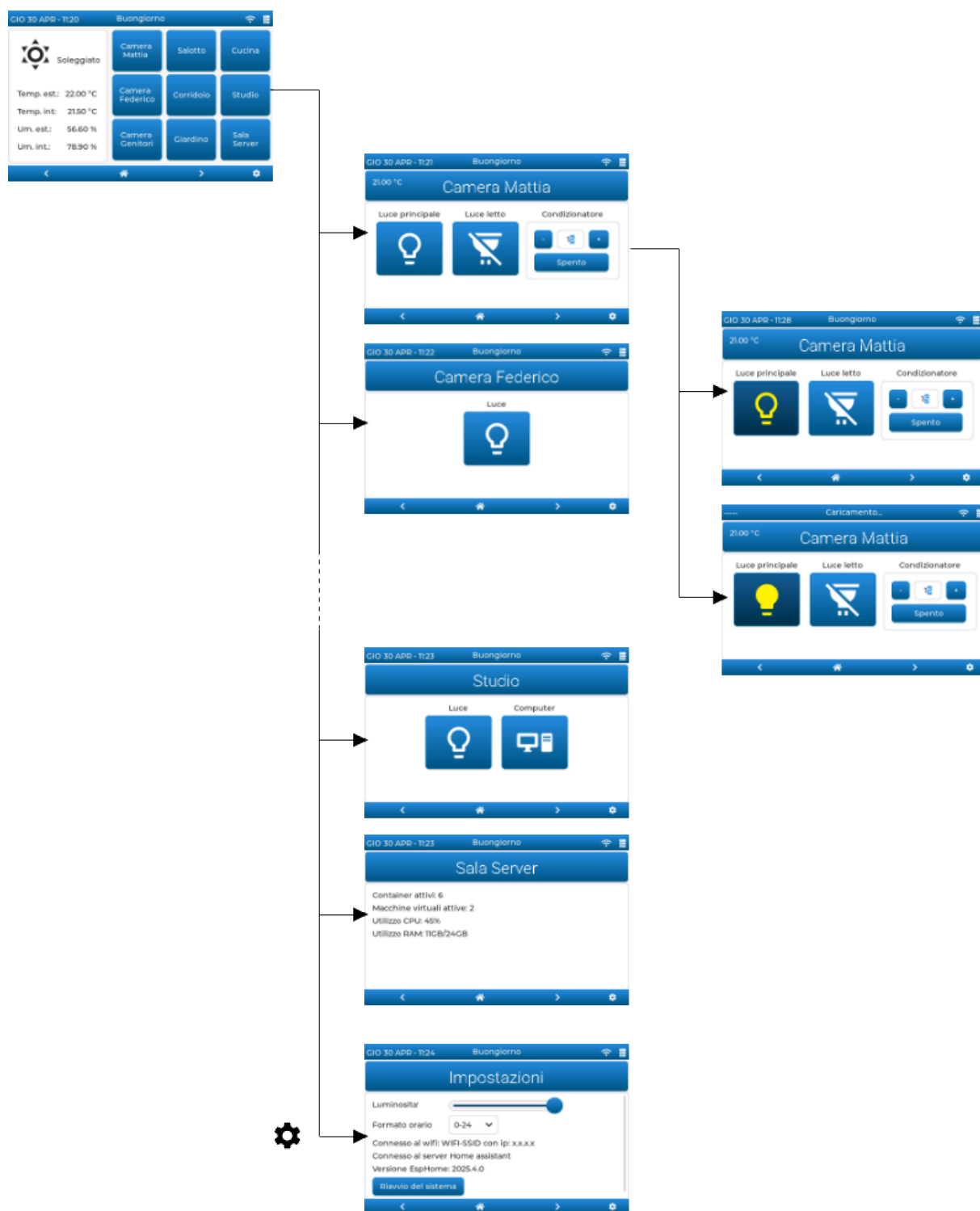


Figura 4.12: Albero di navigazione dell'interfaccia grafica

4.2.3.2 Back-end

L'aggiornamento dell'interfaccia grafica si basa sull'azione *lvgl.label.update*, che permette di aggiornare le informazioni dell'etichetta fornita attraverso il suo id. L'interfaccia grafica risulta ricca di informazioni, le quali sono gestite direttamente dalla scheda che, grazie alla connessione wifi, può accedere ad una serie infinita di risorse.

```

1 wifi:
2   ssid: "FASTWEB-MVF"
3   password: **redacted**
4   on_connect:
5     then:
6       - lvgl.widget.show: wifi_status
7       - delay: 1s
8       - lvgl.label.update:
9         id: lbl_wifi_status
10        text: !lambda return "Connesso al Wifi " + id(wifi_ssid).state + " con ip " + id(
11          ip_addr).state;
12   on_disconnect:
13     then:
14       - lvgl.widget.hide: wifi_status
15       - lvgl.label.update:
16         id: lbl_wifi_status
17         text: "Non connesso al Wifi!"

```

Le chiavi *on_connect* e *on_disconnect* permettono di eseguire delle azioni quando il dispositivo si connette/disconnette al wifi come:

- Mostrare o nascondere l'icona di connessione nella barra di stato 4.7.
- Aggiornare l'etichetta *lbl_wifi_status* con le informazioni sullo stato di connessione visibili nella pagina di Impostazioni 4.11

Per quanto detto nella sezione 2.4 Sicurezza, le risorse sopra citate verranno raccolte dal server Home Assistant utilizzando le API specificate nella sezione 3.3.

```

1 api:
2   encryption:
3     key: **redacted**
4   on_client_connected:
5     - if:
6       condition:
7         lambda: 'return (0 == client_info.find("Home Assistant "));'
8       then:
9         - lvgl.widget.show: lbl_hastatus
10        - lvgl.label.update:
11          id: lbl_hass_status
12          text: !lambda return "Connesso al server Home assistant";
13   on_client_disconnected:
14     - if:
15       condition:
16         lambda: 'return (0 == client_info.find("Home Assistant "));'

```

```

17     then:
18         - lvgl.widget.hide: lbl_hastatus
19         - lvgl.label.update:
20             id: lbl_hass_status
21             text: !lambda return "Non connesso a Home assistant";

```

Qui le chiavi *on_client_connected* e *on_client_disconnected* permettono di eseguire delle azioni quando il dispositivo si connette/disconnette al server Home Assistant come:

- Mostrare o nascondere l'icona di connessione nella barra di stato 4.7
- Aggiornare l'etichetta *lbl_hass_status* con le informazioni sullo stato di connessione visibili nella pagina di Impostazioni 4.11

L'orario e la data vengono sempre forniti da home assistant attraverso l'integrazione *time*

```

1 time:
2   - platform: homeassistant
3     id: htime
4     on_time_sync:
5       - script.execute: time_update
6     on_time:
7       - minutes: '*'
8         seconds: 0
9       then:
10         - script.execute: time_update
11         - script.execute: weather_update

```

Che si occupa di aggiungere il sensore *time* con id *htime* e di gestire l'esecuzione di script per aggiornare le informazioni sulla data e sul meteo.

L'integrazione *script* permette di aggiungere determinate sequenze di codice che possono essere eseguite attraverso l'azione *script.execute: *nome_script**.

1. **time_update**: si occupa dell'aggiornamento dell'etichetta che presenta data e ora come in figura 4.8

```

1 script:
2   - id: time_update
3     then:
4       - lvgl.label.update:
5           id: greetings
6           text: !lambda |-
7               if (!id(htime).now().is_valid()) {
8                   return "Salve";
9               }
10
11               int hour = id(htime).now().hour;
12               if (hour < 12) {
13                   return "Buongiorno";
14               } else if (hour < 18) {
15                   return "Buon pomeriggio";
16               } else if (hour < 22) {

```

```

17         return "Buonasera";
18     } else {
19         return "Buonanotte";
20     }
21     - lvgl.label.update:
22         id: datetime
23         text: !lambda |-
24             static const char * const day_names[] = {"LUN", "MAR", "MER", "GIO", "VEN",
25 "SAB", "DOM"};
26             static const char * const mon_names[] = {"GEN", "FEB", "MAR", "APR", "MAG",
27 "GIU", "LUG", "AGO", "SET", "OTT", "NOV", "DIC"};
28             static char datetime_buf[32];
29
30             auto now = id(htime).now();
31             if (!now.is_valid()) return "----";
32
33             int hour = now.hour;
34
35             // Check the value of time_format and adjust hour accordingly
36             if (id(time_format) == 0) { // 12-hour format
37                 hour = hour % 12;
38                 if (hour == 0) hour = 12; // Hour 0 is represented as 12 in 12-hour
39                 format
40             }
41
42             snprintf(datetime_buf, sizeof(datetime_buf), "%s %2d %s • %d:%02d",
43 day_names[now.day_of_week - 1],
44 now.day_of_month,
45 mon_names[now.month - 1],
46 hour,
47 now.minute
48 );
49
50         return datetime_buf;
51     - id: weather_update
52     ...

```

utilizzando il sensore *htime* si ottengono le informazioni su giorno, mese, anno e orario, questo permette di utilizzarle per mostrare nell'interfaccia descrizioni utili all'utente

2. **weather_update**: si occupa dell'aggiornamento delle informazioni della sezione meteo nella Home Page 4.9

```

1 script:
2     - id: time_update
3     ...
4     - id: weather_update
5     then:
6         - lvgl.label.update:
7             id: weather_outdoor_now
8             text: !lambda |-
9                 char buf[10];
10                 snprintf(buf, sizeof(buf), "%.2f °C", id(current_temp).state);
11                 return std::string(buf);

```



```

12
13     - lvgl.label.update:
14         id: weather_forecast_humidity
15         text: !lambda |-
16             char buf[10];
17             snprintf(buf, sizeof(buf), "%.2f %%", id(forecast_humidity).state);
18             return std::string(buf);
19
20     - lvgl.label.update:
21         id: weather_forecast_condition_name
22         text: !lambda |-
23             std::string cond = id(forecast_condition).state;
24             if (cond == "clear-night") return "Notte serena";
25             if (cond == "cloudy") return "Nuvoloso";
26             if (cond == "exceptional") return "Eccellente";
27             if (cond == "fog") return "Nebbioso";
28             ...
29
30     - lvgl.label.update:
31         id: weather_forecast_condition_icon
32         text: !lambda |-
33             std::string cond = id(forecast_condition).state;
34             if (cond == "clear-night") return "\U000F0594";
35             if (cond == "cloudy") return "\U000F0590";
36             if (cond == "exceptional") return "\U000F0F2F";
37             if (cond == "fog") return "\U000F0591";
38             ...

```

In questo script si fa un elevato uso di icone.

L'integrazione font permette di aggiungere font specifici e icone, affinché possano essere utilizzate per meglio rappresentare le informazioni.

```

1 font:
2   - file: "fonts/materialdesignicons-webfont.ttf"
3     id: icons_70
4     size: 70
5     bpp: 4
6     glyphs: [
7         "\U000F0594", # clear-night
8         "\U000F0590", # cloudy
9         "\U000F0F2F", # exceptional
10        "\U000F0591", # fog
11        ...
12    ]
13    ...

```

Attraverso l'etichetta *glyphs* è possibile aggiungere icone presenti nel font, pronte per essere utilizzate.

I componenti elettronici, citati nella sezione 4.2.2.1, devono essere integrati all'interno di EspHome e seguono l'assegnazione dei pin come nella figura 4.3

Il display si inizializza attraverso l'integrazione *display*

```
1 display:
2   - platform: ili9xxx
3     id: display_1
4     model: ST7796
5     spi_id: lcd
6     dc_pin: GPIO13
7     reset_pin: GPIO20
8     cs_pin: GPIO14
9     invert_colors: true
10    update_interval: 5s
11    auto_clear_enabled: false
12    show_test_card: false
13    dimensions:
14      width: 480
15      height: 320
16    transform:
17      swap_xy: true
18      mirror_x: true
19      mirror_y: true
```

Affinché il display possa comunicare con il microcontrollore è necessario definire l'integrazione *spi*

```
1 spi:
2   - id: lcd
3     clk_pin: GPIO11
4     mosi_pin: GPIO12
5     miso_pin: GPIO9
```

Il touchscreen invece, viene importato grazie all'integrazione *touchscreen*

```
1 touchscreen:
2   - platform: ft63x6
3     i2c_id: touch
4     update_interval: 30ms
5     threshold: 20
6     address: 0x38
7     touch_timeout: 1s
8     interrupt_pin: GPIO18
9     reset_pin: GPIO3
10    id: touchscreen_1
11    transform:
12      swap_xy: true
13      mirror_x: true
14    ...
```

I sensori invece, vengono divisi in tre diverse integrazioni:

- **sensor**: permette di integrare sensori generali
- **binary_sensor**: permette di integrare sensori binari, ovvero che possono avere solo due stati (0/1)
- **text_sensor**: permette di integrare sensori la cui unica caratteristica è quella di contenere al suo interno del testo

Il sensore ambientale viene importato grazie all'integrazione *sensor* che fa utilizzo della piattaforma *bme280_i2c*

```

1 sensor:
2     ...
3     - platform: bme280_i2c
4       i2c_id: bme
5       temperature:
6         name: "Temperature"
7         oversampling: 16x
8         id: temperature
9         on_value:
10          - lvgl.label.update:
11              id: internal_temp
12              text: !lambda |-
13                  char buf[10];
14                  snprintf(buf, sizeof(buf), "%.2f °C", id(temperature).state);
15                  return std::string(buf);
16       pressure:
17         name: "Pressure"
18         id: pressure
19       humidity:
20         name: "Relative Humidity"
21         oversampling: 16x
22         id: humidity
23         on_value:
24          - lvgl.label.update:
25              id: internal_humidity
26              text: !lambda |-
27                  char buf[10];
28                  snprintf(buf, sizeof(buf), "%.2f %%", id(humidity).state);
29                  return std::string(buf);
30       address: 0x76
31       update_interval: 60s

```

La piattaforma possiede tre sottochiavi: *temperature*, *pressure* e *humidity*, che permettono di aggiungere le informazioni riguardanti temperatura, pressione e umidità raccolte dal sensore bme280 nella sezione meteo, come nella figura 4.9

Il touchscreen e il sensore ambientale bme280 comunicano attraverso l'interfaccia *I2C*, di seguito la sua integrazione

```

1 i2c:
2     - id: touch
3       sda: GPIO8
4       scl: GPIO46
5       scan: true
6
7     - id: bme
8       sda: GPIO41
9       scl: GPIO47
10      scan: true

```

L'encoder rotativo invece, viene invece inizializzato attraverso due integrazioni differenti: *sensor* attraverso la piattaforma *rotary_encoder* gestisce la rotazione, mentre *binary_sensor* attraverso la piattaforma *gpio* gestisce il pulsante.

```

1 sensor:
2   ...
3   - platform: rotary_encoder
4     id: rot_encoder
5     pin_a: GPIO5
6     pin_b: GPIO6
7     resolution: 1
8     restore_mode: ALWAYS_ZERO
9     filters:
10      - debounce: 5ms
11     ...
12 binary_sensor:
13   ...
14   - platform: gpio
15     pin:
16       number: GPIO4
17       mode: INPUT_PULLUP
18       inverted: true
19     id: rot_button
20     ...

```

La rotazione del sensore permette di muoversi attraverso l'interfaccia, mentre il pulsante permette di interagire con essa

Anche gli altri due pulsanti, fanno utilizzo dell'integrazione *binary_sensor*:

```

1 binary_sensor:
2   ...
3   - platform: gpio
4     pin:
5       number: GPIO16
6       mode: INPUT_PULLUP
7       inverted: true
8     name: "Button 1"
9     on_press:
10      then:
11        - homeassistant.action:
12              action: light.toggle
13              data:
14                entity_id: light.shellyplus1_a0a3b3dd083c_switch_0
15      ...
16   - platform: gpio
17     pin:
18       number: GPIO17
19       mode: INPUT_PULLUP
20       inverted: true
21     name: "Button 2"
22     on_press:
23      then:
24        - lvgl.page.show: main_page
25      ...

```

E vengono utilizzati come scorciatoie: il pulsante 1 permette di accendere una luce predefinita, ad esempio la luce della stanza in cui si trova il dispositivo; mentre il pulsante 2 permette di tornare alla home page

I sensori di tipo *text_sensor* vengono utilizzati per raccogliere dati dal server di Home Assistant, un esempio di funzionamento viene riportato in seguito:

```

1 text_sensor:
2   ...
3   - platform: homeassistant
4     id: bedroom_mattia_light_main
5     entity_id: light.luci_principali
6     on_value:
7       then:
8         - lvgl.widget.update:
9             id: btn_bedroom_mattia_light_main
10            state:
11              checked: !lambda return (0 == x.compare(std::string{"on"}));
12              disabled: !lambda return ((0 == x.compare(std::string{"unavailable"})) or (0
13              == x.compare(std::string{"unknown"})));
14            - lvgl.label.update:
15              id: lbl_bedroom_mattia_light_main
16              text: !lambda |-
17                static char buf[10];
18                std::string icon;
19                if (0 == x.compare(std::string{"on"})) {
20                  icon = "\U000F0335";
21                } else {
22                  icon = "\U000F0336";
23                }
24                snprintf(buf, sizeof(buf), "%s", icon.c_str());
25                return buf;

```

Questo sensore si occupa di aggiornare due widget nella schermata in figura 4.10:

- Etichetta **lbl_bedroom_mattia_light_main**: cambia l'icona dell'etichetta in relazione allo stato dell'entità raccolto da Home Assistant.
- Pulsante **btn_bedroom_mattia_light_main**: aggiorna gli stati *checked* e *disabled* in relazione allo stato del sensore.

e permettono di realizzare un pulsante che si adatta dinamicamente allo stato dell'entità.



Figura 4.13: Stato dei pulsanti

Capitolo 5

conclusioni

5.1 Ricapitolazione

Negli ultimi 10 anni il mercato della domotica ha visto un aumento repentino di dispositivi intelligente da installare nelle abitazioni, questo porta con se una serie di criticità per quanto riguarda la sicurezza e l'intercompatibilità. Lo sviluppo di software open-source ha permesso di risolvere alcuni di questi problemi, prestando particolare attenzione alla gestione dei dati personali, tenuti lontani dai cloud cercando di offrire soluzioni che permettano di utilizzare i dispositivi con server locali. EspHome assieme ad Home Assistant, i due progetti di più grande valore curati dalla Open Home Foundation, permettono di realizzare dispositivi economici, semplici da utilizzare, programmare e che prestano massima attenzione alla privacy. Nasce quindi Dashi, un dispositivo basato proprio su questi progetti, che permette di controllare la propria abitazione in modo locale e sicuro. Il tutto è racchiuso all'interno di una scocca stampata in 3D che ospita sensori, display touch e pulsanti, collegati attraverso una PCB personalizzata ad un microcontrollore ESP32, cuore pulsante che si occupa di gestire tutti i componenti in modo efficiente. L'interfaccia, sviluppata grazie ad EspHome, restituisce una usabilità fluida e semplice, grazie alla sua suddivisione in diverse pagine, dove ad ognuna di essa è associata una stanza specifica dell'abitazione, la quale rappresenta al suo interno tutti i dispositivi intelligenti in essa presenti. Grazie ad Home Assistant, che integra una serie quasi infinita di dispositivi, è possibile mettere in comunicazione i due progetti, affinché questi possano lavorare in sintonia per restituire una esperienza completa.

5.2 Considerazioni critiche

Nel mercato sono presenti molti prodotti simili, come NS Panel e NS Panel pro prodotti da Sonoff o come Wall Display e Wall Display X2 prodotti da Shelly, con funzioni molto simili

a Dashi e con la possibilità di integrare al suo interno i sensori di Home Assistant, anche Tuya produce una vasta gamma di prodotti simili.

Analizzando i costi, rispetto ai dispositivi commerciali, questi si discostano del 10/20%, rendendo quindi Dashi non competitivo dal punto di vista economico, che può essere inteso come un prodotto che gravita attorno alla privacy dell'utente e si distingue per facilità di personalizzazione.

Lo sviluppo del dispositivo e in particolare la scelta dei componenti che lo compongono è stata limitata dalla scelta dell'hardware supportato da EspHome, non sempre completa. Questa criticità può essere risolta sviluppando moduli personalizzati per EspHome, che porta con sé una serie di criticità che necessitano di una precisa comprensione e analisi dei relativi datasheet.

Altra grande limitazione, dovuta alla scelta di utilizzare EspHome, riguarda la configurazione yaml, che risulta sì semplice nella scrittura e lettura, ma rende alcune operazioni non sempre possibili. Alcune di queste possono essere ovviate utilizzando le funzioni LAMBDA, che rendono però il codice più articolato. Il trade-off tra semplicità di utilizzo e di tutte queste limitazioni risulta comunque favorevole per quello che è stato esposto in questa tesi.

Dashi gravita attorno a progetti open-source, supportati spesso da associazioni no-profit o da sviluppatori indipendenti, questo fa sì che queste realtà possano continuare il loro lavoro solamente se sostenuti da donazioni o da sponsor e rischiano pertanto di terminare le loro attività da un momento all'altro, rischiando di rendere i loro software obsoleti in poco tempo. La sicurezza di progetti open-source è spesso affidata agli utenti stessi, che possono analizzare il codice e scovarne le criticità. Questo, in teoria, garantisce una buona robustezza del codice, ma non sempre tutti gli utenti che lo utilizzano possiedono le capacità necessarie per fare ciò.

5.3 sviluppi futuri

Prendendo spunto dai dispositivi simili, si può notare come questi integrino al loro interno un relè per il controllo di un carico, solitamente un punto luce, e siano alimentati direttamente con tensione 230V. Questo richiederebbe la realizzazione di un circuito di alimentazione in grado di gestire la tensione domestica, sviluppo che comporta una serie di rischi per la persona da valutare con cura. L'integrazione invece di un relè risulterebbe invece più semplice, a patto di una migliore gestione dello spazio all'interno della scocca o una sua eventuale rivisitazione.

Altra soluzione per ottimizzare le dimensioni, oltre che i costi, potrebbe essere quella di realizzare una PCB completa, senza dunque l'utilizzo di breakout board o development board. Questo porterebbe ad una diminuzione drastica dell'ingombro interno, oltre ad una migliore disposizione dei componenti. Questo richiede però abilità di progettazioni avanzate.

Non è sempre possibile installare il dispositivo a parete, sarebbe dunque molto utile realizzare un supporto che permetta di posizionare Dashi su di una superficie piana, affinché possa essere utilizzato ad esempio sopra un comodino o un tavolo.

Bibliografia

- [1] «Istituto della Enciclopedia Italiana fondata da Giovanni Treccani.» (2012), indirizzo: [https://www.treccani.it/enciclopedia/domotica_\(Lessico-del-XXI-Secolo\)/](https://www.treccani.it/enciclopedia/domotica_(Lessico-del-XXI-Secolo)/).
- [2] X10. «About us.» (2013), indirizzo: <https://www.x10.com/pages/about-us>.
- [3] S. Benedict, «x-automate: Control Your Home with Linux,» *linuxjournal*, 1999. indirizzo: <https://www.linuxjournal.com/article/2504>.
- [4] J. Greichen, «Value based home automation for todays' market,» *IEEE Transactions on Consumer Electronics*, vol. 38, n. 3, 1992.
- [5] D. C. Khedekar, A. C. Truco, D. A. Oteyza e G. F. Huertas, «Home Automation—A Fast - Expanding Market,» *Thunderbird International Business Review*, vol. 59, n. 1, 2017. indirizzo: <https://onlinelibrary.wiley.com/doi/abs/10.1002/tie.21829>.
- [6] Martin, «Entrepreneurial insights. Retrieved from TheSmart Home | Intelligent Home Automation,» *CBinsights*, 2014. indirizzo: <http://www.cbinsights.com/blog/home-automation-venture-capital-exits>.
- [7] F. Staff, «Internet of Things, privacy and security in a connected world,» *FTC*, 2015. indirizzo: <https://www.ftc.gov/system/files/documents/reports/federal-trade-commission-staff-report-november-2013-workshop-entitled-internet-things-privacy/150127iotrpt.pdf>.
- [8] E. Fernandes, A. Rahmati, J. Jung e A. Prakash, «Security Analysis of Emerging Smart Home Applications,» in *IEEE Symposium on Security and Privacy*, 2016.
- [9] V. Chang, P. Chundury e M. Chetty, «“Spiders in the Sky”: User Perceptions of Drones, Privacy, and Security,» in *CHI Conference on Human Factors in Computing Systems (CHI)*, 2017.
- [10] K. Hill, «How a Creep Hacked a Baby Monitor to Say Lewd Things to a 2-Year-Old,» *Forbes*, 2013. indirizzo: <https://www.forbes.com/sites/kashmirhill/2013/08/14/>.

- [11] L. Wamsley, «Amazon Echo Recorded And Sent Couple's Conversation — All Without Their Knowledge,» in *NPR*, 2018. indirizzo: <https://www.npr.org/sections/thetwo-way/2018/05/25/>.
- [12] H. Chung, J. Park e S. Lee, «Digital Forensic Approaches for Amazon Alexa Ecosystem,» in *Digital Investigation*, vol. 22, 2017.
- [13] V. Miori, L. Tarrini, M. Manca e G. Tolomei, «An open standard solution for domotic interoperability,» *IEEE Transactions on Consumer Electronics*, vol. 52, n. 1, 2006.
- [14] Samsung. «SmartThings.» (), indirizzo: <https://www.samsung.com/us/smartthings/>.
- [15] Google. «Google Home.» (), indirizzo: https://home.google.com/intl/it_it/about-google-home/.
- [16] W. Zegeye, A. Jemal e K. Kornegay, «Connected Smart Home over Matter Protocol,» in *2023 IEEE International Conference on Consumer Electronics (ICCE)*, 2023.
- [17] I. Unwala, Z. Taqvi e J. Lu, «Thread: An IoT Protocol,» in *2018 IEEE Green Technologies Conference (GreenTech)*, 2018.
- [18] E. Fernandes, J. Jung e A. Prakash, «Security Analysis of Emerging Smart Home Applications,» in *Proceedings of the IEEE Symposium on Security and Privacy*, Piscataway, NJ: IEEE, 2016.
- [19] B. Setz, S. Graef, D. Ivanova, A. Tiessen e M. Aiello, «A Comparison of Open-Source Home Automation Systems,» *IEEE Access*, vol. 9, 2021.
- [20] E. Brown. «Home Assistant: The Python Approach to Home Automation.» (2018), indirizzo: <https://www.linux.com/topic/embedded-iot/home-assistant-python-approach-home-automation/>.
- [21] «Home Assistant Website,» Home Assistant. (), indirizzo: <https://www.home-assistant.io/>.
- [22] «Home Assistant GitHub,» Home Assistant. (), indirizzo: <https://github.com/home-assistant/>.
- [23] P. Schoutsen. «Open Home Foundation.» (2024), indirizzo: <https://www.openhomefoundation.org/>.
- [24] «EspHome,» Open Home Foundation. (), indirizzo: <https://esphome.io/>.
- [25] «Made for EspHome,» EspHome. (), indirizzo: https://esphome.io/guides/made_for_esphome.

- [26] EverythingSmartHome. «Everything Presence One.» (), indirizzo: <https://devices.esphome.io/devices/Everything-Presence-One>.
- [27] «EspHome GitHub repository,» EspHome. (), indirizzo: <https://github.com/esphome/esphome>.
- [28] «ESPHome Developer Documentation,» EspHome. (), indirizzo: <https://developers.esphome.io/architecture/overview/>.
- [29] A. Thomas. «Voluptuous data validation library.» (), indirizzo: <https://github.com/alecthomias/voluptuous>.
- [30] «Platformio Documentation,» Platformio. (), indirizzo: <https://docs.platformio.org/en/latest/what-is-platformio.html>.
- [31] «Cos'è una API,» AWS. (), indirizzo: <https://aws.amazon.com/what-is/api/>.
- [32] «MQTT: The Standard for IoT Messaging,» MQTT organization. (), indirizzo: <https://mqtt.org/>.
- [33] «http request,» World Wide Web Consortium. (), indirizzo: <https://www.w3.org/Protocols/HTTP/Request.html>.
- [34] Google. «Protocol Buffers.» (), indirizzo: <https://protobuf.dev/>.
- [35] EspHome. «EspHome API architecture.» (), indirizzo: <https://developers.esphome.io/architecture/api/>.
- [36] EspHome. «EspHome API component.» (), indirizzo: <https://esphome.io/components/api>.
- [37] M. Recipes. «Umami – The Delicious 5th Taste You Need to Master!» (2013), indirizzo: <http://www.molecularrecipes.com/molecular-gastronomy/umami/>.
- [38] *ESP32-S3-WROOM-1 and ESP32-S3-WROOM-1U*, Version 1.4, Espressif. indirizzo: https://www.espressif.com/sites/default/files/documentation/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf.
- [39] *3.5inch IPS SPI Module ST7796 Datasheet*, LCD wiki. indirizzo: http://www.lcdwiki.com/3.5inch_IPS_SPI_Module_ST7796.
- [40] *KEYES KY-040 Datasheet*, University of Nebraska–Lincoln. indirizzo: <https://eeshop.unl.edu/pdf/KEYES%20Rotary%20encoder%20module%20KY-040.pdf>.
- [41] *BME280 Data sheet*, revision 1.24, Bosch Sensortec. indirizzo: <https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme280-ds002.pdf>.

- [42] Kicad. (2025), indirizzo: <https://www.kicad.org/>.
- [43] «JST 4pin Datasheet,» JLCPCB part. (2025), indirizzo: <https://jlcpcb.com/api/file/downloadByFileSystemAccessId/8588891444421935104>.
- [44] «JST 5pin Datasheet,» JLCPCB part. (2025), indirizzo: <https://jlcpcb.com/api/file/downloadByFileSystemAccessId/8588890104877731840>.
- [45] «PFC Datasheet,» JLCPCB part. (2025), indirizzo: <https://jlcpcb.com/api/file/downloadByFileSystemAccessId/8588945659068497920>.
- [46] «LVGL Github,» LVGL. (2025), indirizzo: <https://github.com/lvgl/lvgl>.
- [47] «LVGL in EspHome,» EspHome. (2025), indirizzo: <https://esphome.io/components/lvgl/index.html>.